

Programming II Final Project Report

Smart Warehouse and Inventory Management System

GitHub Repo

Team:

Elera-Obari Josiah-Chu

Brian Baraza

20th September, 2026

Problem and Users

The problem involves warehouses needing precise records of product locations, available quantities and how stock enters and leaves. Using manual records and spreadsheets causes mistakes such as products placed in the wrong location, incorrect quantities, unnoticed low stock levels and inefficient use of storage space.

A Java desktop application helps warehouse administrators and stock officers manage storage places, products and stock movements through a graphical interface supported by a MySQL database.

Entities and Relationships

Entities	Relationships
storage_location	A self-referencing hierarchy of warehouses, zones and shelves.
products	A product may have inventory records at multiple shelf locations and may have several stock movements.
inventory_items	The link between a product and the location holding it with a quantity
stock_movements	A historical record of stock entering or leaving.

A location may contain several child locations; a product may be stored in multiple locations; each product may have several movement records.

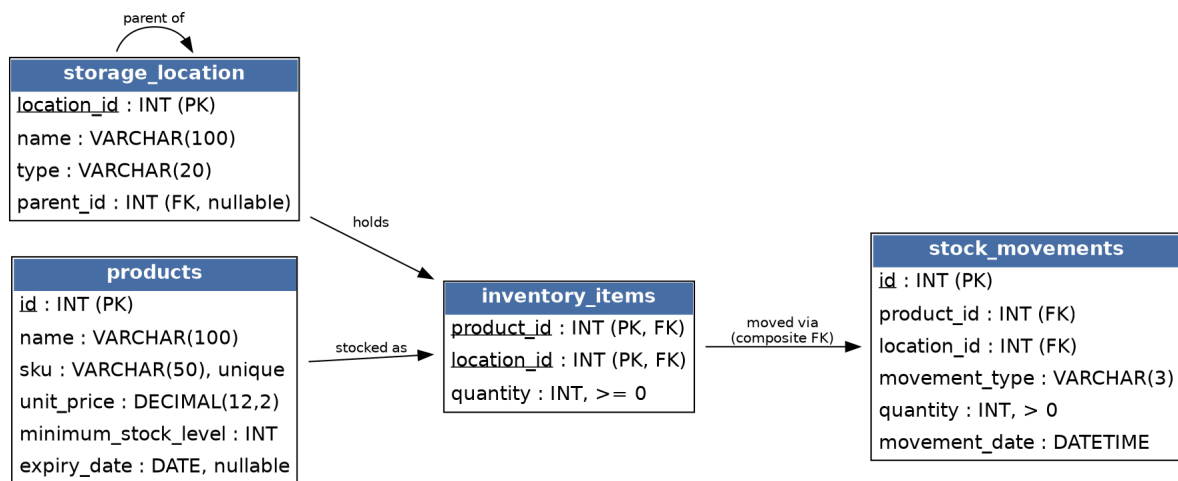


Figure 1: Entity-relationship diagram

CRUD Scope

1. Create, view, update and delete products.
2. Create, view, update and delete storage locations.
3. Automatically assign a product to a shelf during its first stock-in movement and update its quantity through subsequent stock-in and stock-out movements.
4. Record and view stock-in and stock-out movements.
5. Search for a product and identify where it is stored.
6. View low-stock and near-expiry alerts.

Stock movements are treated as immutable historical records. They can be created and viewed through the interface, but they are not updated or deleted because this would alter the inventory audit trail. Products or locations with linked inventory or movement records cannot be deleted, enforced by foreign-key constraints with ON DELETE RESTRICT.

Design Patterns

1. Data Access Object (DAO) Pattern

Application in the system:

The DAO pattern separates database operations from the rest of the application.

ProductDAO, *StorageLocationDAO*, *InventoryItemDAO* and *StockMovementDAO* contain the SQL used to create, retrieve, update and delete application data. SQL for creating the database tables is kept separately in *SchemaInitializer*.

Why it was selected:

This structure keeps SQL out of the service and user interface layers. Service classes handle validation and business rules, while the Swing panels display information and respond to

user actions. Database operations can therefore be changed without redesigning the interface or business logic.

Alternative considered:

An Active Record approach was considered, in which each model class would contain methods such as *save()*, *update()* and *delete()*.

Why the alternative was rejected:

Active Record would mix database responsibilities with domain data. This would make the model classes harder to maintain and conflict with the requirement to isolate SQL from the service and user interface layers.

2. Composite Pattern

Application in the system:

The Composite pattern represents the complete storage hierarchy using one *StorageLocation* class. The *LocationType* value identifies whether a location is a warehouse, zone or shelf, while each *StorageLocation* may contain a list of child locations.

Why it was selected:

A shelf acts as a leaf because it has no children. A warehouse or zone acts as a composite because it may contain other locations. Both can be processed through the same *StorageLocation* type, allowing the recursive stock-value calculation to work uniformly at every level.

Alternative considered:

Separate and unrelated *Warehouse*, *Zone* and *Shelf* classes were considered.

Why the alternative was rejected:

Separate classes would duplicate hierarchy and stock-value logic across three implementations. The Composite pattern provides one consistent structure and allows the same recursive method to calculate totals for a shelf, zone or warehouse.

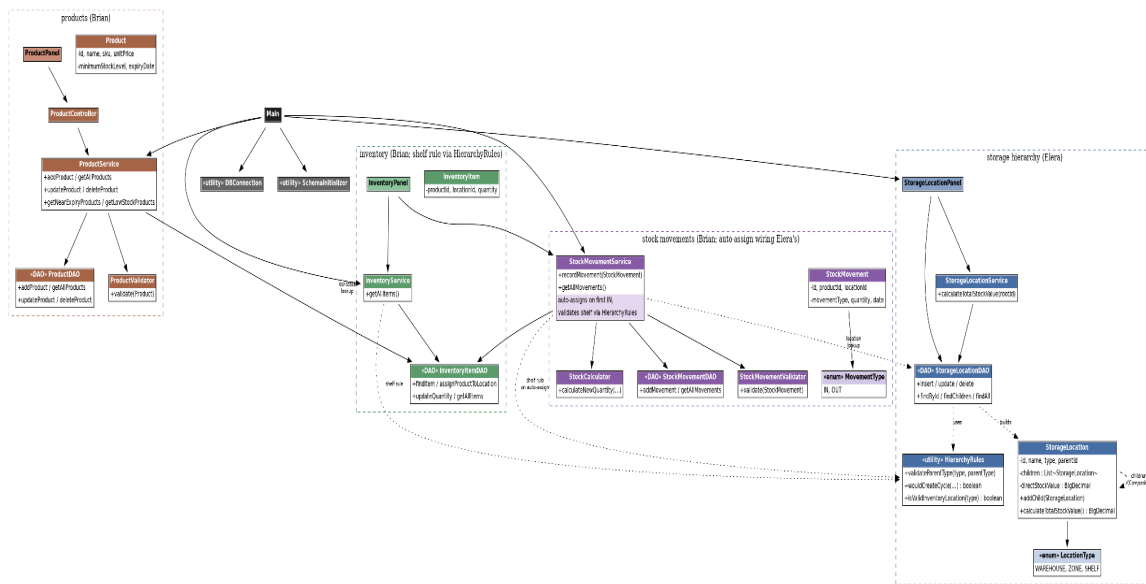


Figure 2. Class diagram grouped by area of ownership

Application of Recursion

The recursive feature is implemented in

StorageLocationService.calculateTotalStockValue(). It calculates the value of stock held at a selected location and at every location below it in the hierarchy.

How the calculation works

For each location, the method adds its direct stock value to the totals returned for all its child locations:

$$\text{Total value of a location} = \text{Direct stock value} + \text{Total value of all child locations}$$

Base case: When a location has no children, the method returns only the stock value held directly at that location. A shelf is normally the base case because it is the lowest level of the hierarchy.

Recursive case: When a location has children, the method calls itself for each child and adds the returned values. A warehouse therefore receives the combined value of its zones, while each zone receives the combined value of its shelves.

Why recursion was selected

Recursion suits this feature because the storage structure is hierarchical. Warehouses contain zones, zones contain shelves, and each level can be processed using the same calculation. An iterative solution would require a manually managed stack or queue to track unvisited locations, effectively reproducing the work already handled by the recursive call stack.

Termination and safety

The recursion cannot continue indefinitely. The hierarchy is limited to three levels: warehouse, zone and shelf. *HierarchyRules.validateParentType()* enforces this structure whenever locations are inserted or updated, while *wouldCreateCycle()* prevents a location from becoming its own ancestor. The method therefore always reaches a location with no children and terminates naturally.

Testing

76 unit tests across five test classes, all passing on a clean build (mvn test), covering both team members' business logic:

Test Class	Tests	Covers
HierarchyRulesAndRecursionTest	13	Recursion base case and depth, parent-type validation, cycle prevention
ProductValidatorTest	32	Product field validation
StockMovementValidatorTest	11	Stock movement validation
StockCalculatorTest	12	Quantity calculation for IN/OUT movements
InventoryValidatorTest	8	Inventory validation, including valid IDs and non-negative quantities

Data entered through the interface persists across a restart verified against a live MySQL database not a mocked connection and the schema is created automatically on first run through *SchemaInitializer.initAll()* in foreign-key-safe order.

Build and Launch Instructions

Build and Launch Summary

1. Database setup: Create a MySQL database named *warehouse_db*.
2. Configuration: Add a *db.properties* file in the project root containing the database URL, username and password.
3. Build: Run *mvn clean package* to compile the application, execute all unit tests and generate the executable JAR.
4. Packaging: The application was packaged using *jpacakge*. The package includes a bundled Java Runtime Environment, so the user does not need to install Java separately.

5. Verification: The packaged application was successfully tested on Windows 11 Pro, version 25H2.

Full database setup, build and launch instructions are provided in [README.md](#).

Individual Contributions

Elera-Obari Josiah-Chu

I implemented:

1. Storage-location hierarchy: StorageLocation composite class, LocationType enum
2. StorageLocationDAO with full CRUD, parameterized queries and validation against invalid parent types and cycles (HierarchyRules)
3. Recursive total-stock-value calculation and its unit tests
4. Database schema and its automatic creation on startup (SchemaInitializer)
5. Database connection layer
6. Storage Locations GUI screen
7. Packaging (jpackage)
8. The integration point where a product is automatically assigned to a shelf on its first stock-in movement, including the shelf-only validation shared between InventoryService and StockMovementService

Brian Baraza

I implemented:

1. Product management model, validation, DAO, service and controller classes.
2. Product GUI functions for creating, viewing, updating, deleting, searching and refreshing products.
3. Low-stock and near-expiry detection and their related interface filters.
4. Inventory item model, DAO, service, validation and inventory display.
5. Stock movement model, movement type, DAO, service, validation and stock calculator.
6. Stock-in and stock-out processing, movement history and prevention of stock quantities falling below zero.

7. Transaction handling so that quantity updates and movement records either both succeed or both roll back.
8. Sixty-three unit tests covering product validation, inventory validation, stock-movement validation and IN/OUT quantity calculations.
9. Integration testing of product, inventory and movement functions against the live database, including persistence and restricted deletion.
10. Storage-location GUI update support to complete the second full CRUD interface.